实验报告

课程名称：Design pattern and software architecture

学　　院：计算机科学与工程学院

专　　业:Computer Science and Technology　班　　级：SE-1

姓　　名：Muhammad Samudera　学　　号：

年　　月　　日

山东科技大学教务处制

实 验 报 告

组　别	1	姓　名	Samudera	同组实验者	None
实验项目名称	Composite Pattern				
教师评语	Good understanding of the pattern idea and implementation. The report is clear and written in proper English.				
实验成绩:		指导教师（签名）： 2026 3 23			

1. Objective

The objective of this experiment is to implement the Composite design pattern using an order pricing system example involving OrderComponent (Component), Product (Leaf), and Box (Composite). The goal of this pattern is to compose objects into tree structures to represent part-whole hierarchies, allowing clients to treat individual objects and compositions of objects uniformly.

In this example, the program has the following hierarchy:

- **OrderComponent** (Component interface): defines the common interface shared by both individual items and composite containers. It declares the getPrice() method that every element in the order tree must implement.
- **Product** (Leaf): represents a single, indivisible item in the order (e.g., a book, pen, notebook, or ruler). It implements OrderComponent and returns its own price directly.
- **Box** (Composite): a container that can hold other OrderComponents — both individual Products and other Boxes. It implements OrderComponent by recursively summing the prices of all its children.
- **Client**: acts as the consumer that interacts exclusively with the OrderComponent interface. It does not need to know whether the underlying object is a single Product or a nested Box — the Composite pattern makes them interchangeable.

2. Principle

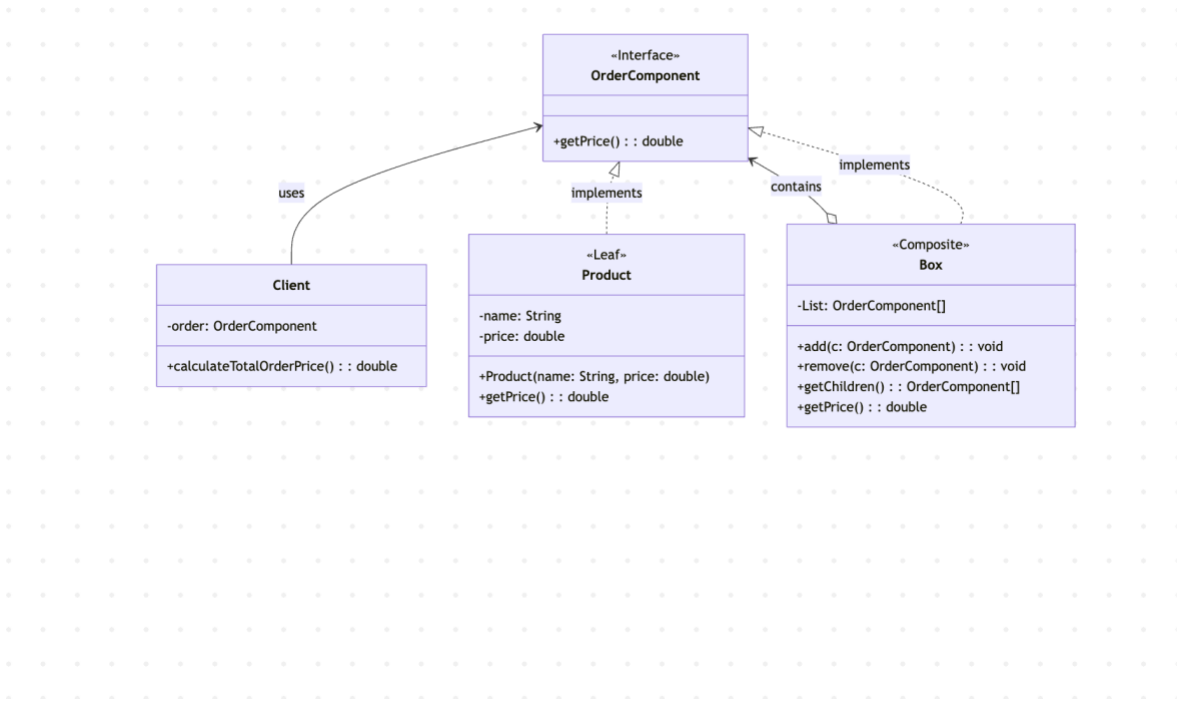
The Composite pattern composes objects into tree structures to represent part-whole hierarchies. It lets clients treat individual objects (Leaf) and compositions of objects (Composite) uniformly through a common interface.

Instead of the client having to distinguish between a single product and a box full of products, the Composite pattern defines a shared interface (OrderComponent) that both Product and Box implement. When the client calls getPrice(), the call is handled transparently: a Product returns its own price, while a Box recursively aggregates the prices of all its children. This recursive composition enables arbitrarily deep nesting — a box can contain other boxes, which in turn contain products or even more boxes.

Key benefits of this approach include:

- Uniformity: The client treats individual products and composite boxes through the same OrderComponent interface, eliminating the need for type-checking or conditional logic.
- Recursive composition: Boxes can contain other Boxes, enabling the construction of complex, deeply nested order structures without changing any existing code.
- Open/Closed Principle: New component types (e.g., a DiscountedProduct or a GiftBox) can be added by implementing the OrderComponent interface without modifying existing classes.
- Simplified client code: The Client class only interacts with OrderComponent, remaining completely unaware of whether the order is a single item or an entire tree of nested boxes.
- Transparent pricing: Price calculation is delegated recursively through the tree, so the total price of any subtree or the entire order is computed through a single getPrice() call.

3. UML Diagram



4. Key Code (Java)

```
nterface OrderComponent {                                    // Component
    double getPrice();
}


class Product implements OrderComponent {                    // Leaf
    private String name;
    private double price;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
```

```
    }

    public double getPrice() {
        return price;
    }

    public String toString() {
        return "Product[name=" + name + ", price=" + price + "]";
    }
}

class Box implements OrderComponent {                    // Composite
    private List<OrderComponent> children = new ArrayList<>();

    public void add(OrderComponent c) {
        children.add(c);
    }

    public void remove(OrderComponent c) {
        children.remove(c);
    }
```

```
    public List<OrderComponent> getChildren() {
        return children;
    }

    public double getPrice() {                          // Recursive aggregation
        double total = 0;
        for (OrderComponent child : children) {
            total += child.getPrice();
        }
        return total;
    }
}

class Client {
    private OrderComponent order;

    public Client(OrderComponent order) {
        this.order = order;
    }

    public double calculateTotalOrderPrice() {
        return order.getPrice();
```

```
    }
}


// Client usage — treating Leaf and Composite uniformly
Product book = new Product("Book", 25000);
Product pen = new Product("Pen", 5000);
Product notebook = new Product("Notebook", 15000);
Product ruler = new Product("Ruler", 8000);


Box smallBox = new Box();
smallBox.add(pen);
smallBox.add(notebook);


Box mainBox = new Box();
mainBox.add(book);
mainBox.add(smallBox);        // nesting a box inside another box
mainBox.add(ruler);


Client client = new Client(mainBox);
client.calculateTotalOrderPrice();   // returns 53000.0 — uniform traversal of entire tree
```

5. Analysis

- **Advantages:** The Composite pattern provides a powerful mechanism for building recursive, tree-structured data models while keeping the client interface simple and uniform. In this program, the Client class interacts only with the OrderComponent interface

and calls a single method — getPrice() — regardless of whether the order consists of one product or a deeply nested tree of boxes. This eliminates the need for conditional logic or type-checking on the client side. Adding a new type of component, such as a DiscountedProduct or a GiftWrapBox, only requires implementing the OrderComponent interface; no existing classes need to be modified, adhering to the Open/Closed Principle. The recursive nature of Box.getPrice() means that any level of nesting is handled automatically: a box within a box within a box will still compute its total price correctly through delegation.

- **Disadvantages:** The Composite pattern can make it harder to restrict the composition to allow only certain types of children. For instance, in this system, there is no compile-time enforcement to prevent a Product from being added to another Product (since the add method is only on Box, this particular issue is avoided, but in designs where add/remove are defined on the Component interface, this becomes a real concern). Additionally, the uniform interface can sometimes be too general — operations that only make sense for composites (like add and remove) do not apply to leaves, which may require runtime checks or empty implementations. The recursive traversal can also make debugging more complex, since a single getPrice() call may trigger a deep chain of delegated calls across multiple levels of the tree. Finally, for very simple scenarios where the hierarchy is flat and unlikely to grow, the pattern introduces unnecessary indirection.

6. Conclusion

The Composite pattern is well-suited for situations where objects need to be organized into tree structures and treated uniformly, such as in this order pricing system. In this program, OrderComponent defines the common interface, Product represents individual items (leaves), and Box acts as a container (composite) that recursively aggregates the prices of its children. By composing OrderComponents into a tree, the system allows arbitrarily complex order structures to be priced through a single getPrice() call.

This approach is demonstrated directly in the program output: individual products (Book: Rp 25000, Pen: Rp 5000, Notebook: Rp 15000, Ruler: Rp 8000) and composite containers (Small Box: Rp 20000) are all priced through the same interface, and the total order price (Rp 53000) is computed by traversing the entire tree. The Client class interacts solely with OrderComponent, remaining completely decoupled from the internal structure of the order.

Overall, the Composite pattern is a practical and scalable solution for managing hierarchical, part-whole relationships. It eliminates the need for the client to distinguish between individual objects and compositions, promotes code reuse through recursive delegation, and keeps the system maintainable as new component types are added. When the domain naturally forms a tree structure such as file systems, UI component trees, or order/product hierarchies the Composite pattern serves as an essential structural building block in software architecture.